MATH50011 Coursework 1: Continuous Normalizing Flows and Maximum Likelihood Training in Generative Modeling

02556173

February 7, 2026

Introduction

At its core, generative modeling is the statistical task of learning the underlying distribution of an observed dataset for two purposes: generating new samples with realistic characteristics resembling the original data, and evaluating how likely an observation is under the model. To this day, various machine learning paradigms have been developed to achieve these objectives, each with its own source of inspiration, architecture, and training methodology, with variational autoencoders (VAEs) and generative adversarial networks (GANs) amongst the most popular. The focus of this report is a flow-based model: continuous normalizing flows (CNFs). Intuitively, a CNF treats the density of a distribution as a fluid that evolves over time according to an ordinary differential equation defined by a neural network (a neural ordinary differential equation, or NODE).

In flow-based models, we can explicitly learn the data distribution and compute its density (and hence its log-likelihood) in a tractable manner, allowing them to be trained via maximum likelihood estimation (MLE). This differs from VAEs, which optimize a lower bound on the log-likelihood (the ELBO) because the true posterior is unknown in the defining latent-variable framework, and from GANs, whose training procedure is game-theoretic (a minimax game) and does not involve an explicit density at all [1].

This first coursework will be mainly theory-focused, and we will explore:

- the theoretical foundation of flow-based models, including normalizing flows, which is a discretized precursor of CNFs,
- how flow-based models are trained via likelihood-based methods, and
- methods to improve their modeling efficiency and the statistical implications of these methods.

Normalizing flows: a precursor

Normalizing flows (NFs) are generative models that learn an invertible transformation which maps a simple base distribution (e.g. a standard Gaussian) to a distribution matching the observed or target data. This invertibility implies that no information should be lost during the forward and backward passes between the data space (where our target distribution lives) and the latent space (where our base distribution lives), so that exact sampling and inference are possible. This is not the case in VAEs, for example, where the latent dimensionality is often much smaller than its data counterpart for facilitating feature extraction, which may lead to detail loss in case of excessive downsizing.

More importantly, the bijectivity in NFs allows us to use the classic *change of variables* formula to calculate exact densities and subsequently perform MLE as the training method.

Theorem 1 (Change of variables)**.** *Let $f : \mathbb{R}^d \to \mathbb{R}^d$ be bijective and sufficiently smooth, and let $Y = f(X)$, where X is some random variable with support in $\mathbb{R}^d$ and density $p_X : \mathbb{R}^d \to [0, \infty)$. Then the density of Y satisfies*

$$p_Y(y) = p_X\big(f^{-1}(y)\big) \left| \det \frac{\partial f}{\partial x}\big(f^{-1}(y)\big) \right|^{-1}.$$

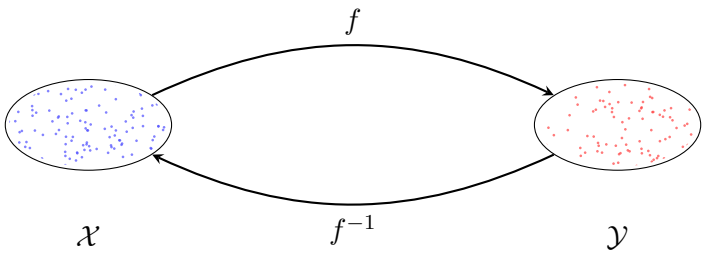


Figure 1: A bijection between latent space $\mathcal{X}$ and data space $\mathcal{Y}$ (hosting X and Y, respectively).

Proof. Say X takes values in $S \subseteq \mathbb{R}^d$, and Y has support $R \coloneqq f(S) \subseteq \mathbb{R}^d$. Due to properties of Jacobian determinants, it suffices to show $p_Y(y) = p_X\left(f^{-1}(y)\right)\left|\det \frac{\partial x}{\partial y}\right|$. Indeed, for any $B \subseteq R$,

$$P(Y \in B) = P\left(X \in f^{-1}(B)\right) = \int_{f^{-1}(B)} p_X(x)\,\mathrm{d}x$$

and by substituting $x = f^{-1}(y)$ into the integral, we have $P(Y \in B) = \int_B p_X(f^{-1}(y))\left|\det \frac{\partial x}{\partial y}\right| \mathrm{d}y$ as desired [3]. (Intuitively, $\mathrm{d}x = \left|\det \frac{\partial x}{\partial y}\right| \mathrm{d}y$ reveals the role of the Jacobian determinant as a volumetric scale factor under the transformation.) □

In the canonical setup of NFs, $X \sim \mathcal{N}(0, I_d)$ is a simple base distribution on the latent space $\mathcal{X}$, and Y is the (unknown) data-generating distribution on $\mathcal{Y}$ from which we observe samples $y_1, \ldots, y_n$. Moreover, $f = f_\theta$ is a flexible neural network model (e.g. multi-layer perceptrons (MLPs) or residual networks (ResNets)) parametrized by a set of parameters θ. Training proceeds from the data space back to the latent space: for each observed y, we evaluate the inverse map $x = f_\theta^{-1}(y)$ and compute its log-density under the base Gaussian, $\log p_X(x)$. We then correct for the local volume change using the Jacobian determinant term from Theorem 1, yielding a tractable expression for the *data log-likelihood* $\log p_\theta(y)$. Finally, we optimize θ by stochastic gradient ascent on the average log-likelihood [3]:

$$\max_\theta \mathbb{E}_{y\sim Y}\left[\log p_\theta(y)\right] = \max_\theta \mathbb{E}_{y\sim Y}\left[\log p_X\left(f_\theta^{-1}(y)\right) - \log\left|\det \frac{\partial f_\theta}{\partial x}\left(f_\theta^{-1}(y)\right)\right|\right]. \tag{1}$$

After the MLE training in (1), we would ideally be able to produce approximate samples of Y by simply sampling from $x \sim X$ and evaluating $f(x)$: the $\log p_X(f_\theta^{-1}(y))$ term encourages mapped-back points to land in high-density regions of the base Gaussian X (conducive for generating good quality samples in the forward pass as f is bijective), while the $-\log\left|\det \frac{\partial f_\theta}{\partial x}\left(f_\theta^{-1}(y)\right)\right|$ term acts as a volume scaling cost that penalizes "artificially" moving points into high base-density regions rather than truly learning the structure of the data.

Comment. In the general context of machine learning, being able to train using MLE is considerably beneficial. Under mild regularity conditions (e.g. smoothness of likelihood and identifiability of the statistical model), the maximum likelihood estimator is *consistent*: as the sample size grows, the estimated parameter converges in probability to the true parameter that generated the data [5, 6]. Consequently, likelihood-based training provides a large-sample guarantee: if the model family is correctly specified, MLE recovers the ground-truth distribution; if it is misspecified, MLE converges to the "closest" (in the sense of KL-divergence) model within the family.

Meanwhile, NFs also have important limitations: requiring f to be bijective and its Jacobian determinant to be tractable can significantly constrain the model's expressivity, and training can be computationally expensive because evaluating the log-determinant Jacobian costs typically $O(d^3)$ (using Gaussian elimination or LU decomposition). CNFs, on the other hand, reformulate the transformation f_θ as a continuous-time dynamics and trade Jacobian determinant for (often cheaper) trace computations [3].

Continuous normalizing flows: the theory

The architecture of CNFs is motivated by first trying to solve the expressivity issue of NFs: going from $\mathcal{X}$ to $\mathcal{Y}$ as in Figure 1, one can compose many invertible f_{θ_i} and train this discrete stack of layers of neural nets by passing down the log-likelihood from layer to layer, using Theorem 1 as before. This would indeed enable the model to learn more complex data patterns, but it doesn't address the computational bottleneck caused by needing to evaluate log-determinant Jacobians.

Instead of applying a finite sequence of discrete bijections, CNFs model a continuous-time family of transformations that transports a simple base distribution to a complex target distribution on $\mathbb{R}^d$. Concretely, we consider the following random NODE:

$$y(0) \sim \mathcal{N}(0, I_d), \qquad \frac{\mathrm{d}y}{\mathrm{d}t}(t) = f_\theta\big(t, y(t)\big), \quad t \in [0, T].$$

The training objective now is to choose θ so that the terminal distribution of $y(T)$ (induced by the pushforward from $y(0)$) approximates the data distribution Y [3].

In the spirit of MLE training, the question now becomes: as the state $y(t)$ evolves under this NODE, how does its density $p_\theta(t, y(t))$ evolve? Answering this leads to the *instantaneous change of variables* formula, which only requires a trace operation instead of an expensive log-determinant term. It is worth mentioning that f_θ here does not need to be bijective, as long as it is Lipschitz continuous in y and continuous in t (something which is typically true of a neural net) [3].

Theorem 2 (Instantaneous change of variables)**.** *Assume $y(t) \in \mathbb{R}^d$ is a finite continuous random variable that evolves according to the ODE*

$$\frac{\mathrm{d}y}{\mathrm{d}t}(t) = f_\theta\big(t, y(t)\big) \tag{2}$$

where f_θ is Lipschitz in y and continuous in t, and assume $y(t)$ admits a time-dependent density $p_\theta(t, \cdot)$ (the subscript denotes the dependence on f_θ). Then the density evolves according to the equation

$$\frac{\partial}{\partial t} \log p_\theta\big(t, y(t)\big) = -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}(t, y(t))\right). \tag{3}$$

Proof. We study the infinitesimal limit of finite changes of the log density through time.

Let $T_\varepsilon(y(t)) \coloneqq y(t + \varepsilon)$. Under the stated continuity assumptions about f_θ, by Picard-Lindelöf theorem, every initial value problem to (2) has a unique solution y. T_ε is therefore a local diffeomorphism (bijective and smooth), allowing the change of variables formula to be applied. Since y is bounded, we also know that f_θ, T_ε, and $\frac{\partial}{\partial y} T_\varepsilon$ are bounded on compact time intervals. We will use these conditions to exchange limits and products.

Firstly, Theorem 1 gives

$$p_\theta(t + \varepsilon, T_\varepsilon(y)) = p_\theta(t, y) \left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|^{-1}.$$

Taking logs and rearranging,

$$\frac{\log p_\theta(t + \varepsilon, y(t + \varepsilon)) - \log p_\theta(t, y(t))}{\varepsilon} = -\frac{1}{\varepsilon} \log \left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|.$$

Letting $\varepsilon \to 0^+$ yields

$$\frac{\partial}{\partial t} \log p_\theta(t, y(t)) = -\lim_{\varepsilon \to 0^+} \frac{\log \left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|}{\varepsilon}. \tag{4}$$

As $T_0(y) = y$, $\frac{\partial T_0}{\partial y}(y) = I_d$ and the RHS of (4) is a $\frac{0}{0}$ indeterminate. By L'Hôpital's rule (L) and Jacobi's formula (J),

$$\begin{aligned}
-\lim_{\varepsilon\to 0^+} \frac{\log\left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|}{\varepsilon} &\overset{\text{L}}{=} -\lim_{\varepsilon\to 0^+} \frac{\frac{\partial}{\partial\varepsilon}\left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|}{\left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right|} \\
&= -\lim_{\varepsilon\to 0^+} \frac{\partial}{\partial\varepsilon}\left|\det \frac{\partial T_\varepsilon}{\partial y}(y)\right| \\
&\overset{\text{J}}{=} -\lim_{\varepsilon\to 0^+} \operatorname{tr}\left(\operatorname{adj}\left(\frac{\partial}{\partial y}T_\varepsilon(y)\right)\frac{\partial^2}{\partial\varepsilon\partial y}T_\varepsilon(y)\right) \\
&= -\operatorname{tr}\left(\left(\lim_{\varepsilon\to 0^+}\operatorname{adj}\left(\frac{\partial}{\partial y}T_\varepsilon(y)\right)\right)\left(\lim_{\varepsilon\to 0^+}\frac{\partial^2}{\partial\varepsilon\partial y}T_\varepsilon(y)\right)\right) \\
&= -\operatorname{tr}\left(\lim_{\varepsilon\to 0^+}\frac{\partial}{\partial\varepsilon}\frac{\partial}{\partial y}T_\varepsilon(y)\right).
\end{aligned}$$

Note that we are allowed to drop the absolute value before applying Jacobi's formula as $\det\left(\frac{\partial T_0}{\partial y}(y)\right) = 1 > 0$ and T_ε (hence the determinant) depends continuously on ε.

Using (2) and the Taylor expansion $T_\varepsilon(y) = y(t+\varepsilon) = y + \varepsilon f_\theta(t,y) + O(\varepsilon^2)$, (4) becomes

$$\begin{aligned}
\frac{\partial}{\partial t}\log p_\theta\left(t, y(t)\right) &= -\operatorname{tr}\left(\lim_{\varepsilon\to 0^+}\frac{\partial}{\partial\varepsilon}\left(I_d + \frac{\partial}{\partial y}\varepsilon f_\theta\left(t,y\right) + O\left(\varepsilon^2\right)\right)\right) \\
&= -\operatorname{tr}\left(\lim_{\varepsilon\to 0^+}\left(\frac{\partial}{\partial y}f_\theta\left(t,y\right) + O\left(\varepsilon\right)\right)\right) \\
&= -\operatorname{tr}\left(\frac{\partial}{\partial y}f_\theta\left(t,y\right)\right)
\end{aligned}$$

as desired [2]. □

With Theorem 2, we can now train a CNF by solving (3) and utilizing the same MLE technique we saw before. The general idea is that given $u \in \mathbb{R}^d$, a terminal condition (i.e., an empirical sample $u \in Y$, just like how we started from the data space when training NFs), and $y(t,u)$, a function of t characterized by u, we consider the terminal value problem

$$y(T,u) = u, \qquad \frac{\mathrm{d}y}{\mathrm{d}t}(t,u) = f_\theta(t, y(t,u)), \quad t \in [0,T] \tag{5}$$

which will be solved backwards in time from $t = T$ to $t = 0$ (usually done with a numerical ODE solver like Euler or RK4) [3].

More specifically, given a batch of empirical samples $y_1, \ldots, y_N \in \mathbb{R}^d$, MLE states that with respect to θ, we should minimize the negative average log-likelihood loss

$$-\frac{1}{N}\sum_{i=1}^N \log p_\theta(T, y_i) = -\frac{1}{N}\sum_{i=1}^N \left[\underbrace{\log p_\theta\big(0, y(0,y_i)\big)}_{\text{א}} - \underbrace{\int_0^T \operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}(t, y(t,y_i))\right)\mathrm{d}t}_{\text{ב}}\right] \tag{6}$$

where the equality can be seen from integrating (3).

The right-hand side of (6) can be readily evaluated:

א: Recall from the random NODE that $p_\theta(0,\cdot)$ is the density of a multivariate standard normal, so it remains to evaluate $y(0,y_i)$.

ↄ: Starting from some empirical sample $y_i \in \mathbb{R}^d$, we solve the ODE in (5) backwards in time from $t = T$ to $t = 0$. As the numerical solution progresses, we obtain $y(t, y_i)$ for all $t \in [0, T]$, which are inputs to the integral ↄ and the $y(0, y_i)$ concerned in ℵ. This integral may actually be evaluated as part of this backwards solving, as we can simply concatenate the integral together with (5) to form an augmented (system of) ODE:

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} y(t, y_i) \\ \log p_\theta\big(t, y(t, y_i)\big) \end{bmatrix} = \begin{bmatrix} f_\theta\big(t, y(t, y_i)\big) \\ -\operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}\big(t, y(t, y_i)\big)\right) \end{bmatrix}. \tag{7}$$

This enables an ODE solver to evolve both the sample path and its log-density in one pass, producing directly $y(0, y_i)$ and the accumulated log-density change [1].

Having calculated the loss in (6), it is backpropagated to update the parameters θ in our neural network f_θ via stochastic gradient descent. Note that this backpropagation is a reverse-time procedure progressing from $t = 0$ to $t = T$, because evaluating (6) involves solving backwards. Finally, an adequately and correctly trained CNF should allow us to generate target samples by sampling $y(0) \sim \mathcal{N}(0, I_d)$ and then solve the random NODE forwards to obtain $y(T) \sim Y$ [3].

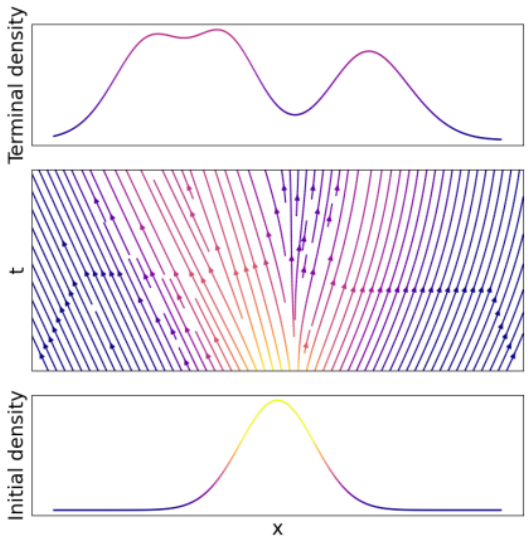


Figure 2: A CNF continuously deforms one distribution into another. The flow lines show how particles from the base distribution are perturbed until they approximate the target distribution (graphics by Patrick Kidger [3]).

Upon revisiting (6) or (7), one may realize that unlike in NFs, we only need to evaluate trace Jacobians here, which is possible via standard autodifferentiation software (reverse-mode). This requires $O(d^2)$ cost, which is indeed cheaper than the log-determinant Jacobian cost in NFs [3]. Nevertheless, we can further improve the evaluation efficiency by deploying *Hutchinson's trace estimator*.

Theorem 3 (Hutchinson, 1989)**.** *If $v = (v_1, \ldots, v_d)^T$ is a random vector of independent entries sampled from a random variable V with mean $\mu = 0$ and variance σ^2, then*

$$\mathbb{E}\left(v^T B v\right) = \sigma^2 \operatorname{tr}(B)$$

for any $B = (b_{ij}) \in \mathbb{R}^{d \times d}$. Moreover, if B is symmetric, then

$$\operatorname{Var}\left(v^T B v\right) = 2\sigma^4 \sum_{i \neq j} b_{ij}^2 + \left(\mathbb{E}\left(V^4\right) - \sigma^4\right) \sum_i b_{ii}^2.$$

Proof. Expanding the quadratic form in expectation, we obtain

$$\begin{aligned}\mathbb{E}\left(v^T B v\right) &= \sum_{1\le i,j\le d} b_{ij}\mathbb{E}(v_i v_j) \\ &= \sum_{1\le i\le d} b_{ii}\mathbb{E}\left(v_i^2\right) \\ &= \left(\mathrm{Var}(V) + \mathbb{E}(V)^2\right)\mathrm{tr}(B) \\ &= \sigma^2\,\mathrm{tr}(B)\end{aligned}$$

where the second and third equality are due to $\mathbb{E}(v_i) = 0$ and v_i being iid.

Now assuming B is symmetric, we have

$$\begin{aligned}\mathbb{E}\left[\left(v^T B v\right)^2\right] &= \mathbb{E}\left[\left(\sum_{i,j} b_{ij} v_i v_j\right)\left(\sum_{r,s} b_{rs} v_r v_s\right)\right] \\ &= \sum_{i,j,r,s} b_{ij} b_{rs}\,\mathbb{E}\left(v_i v_j v_r v_s\right).\end{aligned}$$

Because the entries of v are independent with $\mathbb{E}(v_i) = 0$, the expectation $\mathbb{E}(v_i v_j v_r v_s)$ vanishes unless each index appears an even number of times. Hence the only nonzero contributions come from the following cases:

- $i = r$, $j = s$ with $i \neq j$, and $i = s$, $j = r$ with $i \neq j$;
- $i = j$, $r = s$ with $i \neq r$;
- $i = j = r = s$.

Writing $m_4 \coloneqq \mathbb{E}(V^4)$ and using $\mathbb{E}(v_i^2) = \sigma^2$, we obtain

$$\begin{aligned}\mathbb{E}\left[\left(v^T B v\right)^2\right] &= \underbrace{\sigma^4 \sum_{i\neq j} b_{ij}^2}_{(i=r,\,j=s)} + \underbrace{\sigma^4 \sum_{i\neq j} b_{ij} b_{ji}}_{(i=s,\,j=r)} + \underbrace{\sigma^4 \sum_{i\neq r} b_{ii} b_{rr}}_{(i=j,\,r=s)} + \underbrace{m_4 \sum_i b_{ii}^2}_{(i=j=r=s)} \\ &= 2\sigma^4 \sum_{i\neq j} b_{ij}^2 + \sigma^4 \sum_{i\neq r} b_{ii} b_{rr} + m_4 \sum_i b_{ii}^2,\end{aligned}$$

where the last equality is due to B's symmetry.

On the other hand,

$$\mathbb{E}\left(v^T B v\right) = \sigma^2\,\mathrm{tr}(B) \qquad \Longrightarrow \qquad \mathbb{E}\left(v^T B v\right)^2 = \sigma^4\left(\mathrm{tr}(B)\right)^2 = \sigma^4\left(\sum_i b_{ii}\right)^2.$$

Noting that $\sum_{i\neq r} b_{ii} b_{rr} = \left(\sum_i b_{ii}\right)^2 - \sum_i b_{ii}^2$, we conclude

$$\begin{aligned}\mathrm{Var}\left(v^T B v\right) &= \mathbb{E}\left[\left(v^T B v\right)^2\right] - \mathbb{E}\left(v^T B v\right)^2 \\ &= 2\sigma^4 \sum_{i\neq j} b_{ij}^2 + \left(m_4 - \sigma^4\right)\sum_i b_{ii}^2,\end{aligned}$$

as claimed [4]. □

Remark. Assuming B to be symmetric doesn't harm the generality of the estimator $v^T B v$ in the sense of its variance, since B can always be decomposed as $B = \left(B + B^T\right)/2 + \left(B - B^T\right)/2 =: B_{\mathrm{sym}} + B_{\mathrm{skew}}$, while the quadratic form of a skew-symmetric matrix is identically zero (as $\left(v^T B_{\mathrm{skew}} v\right)^T = -v^T B_{\mathrm{skew}} v$).

Corollary 4. *If $\mu = 0$ and $\sigma^2 = 1$ (e.g. V follows the standard normal or Rademacher distribution), then $v^T Bv$ is an unbiased estimator of $\operatorname{tr}(B)$. Moreover, if V is Rademacher, then $\operatorname{Var}(v^T Bv) = 2\sum_{i \neq j} b_{ij}^2$ and V is unique amongst zero-mean random variables for which $v^T Bv$ is an unbiased estimator with minimal variance.*

Proof. From the expression of $\operatorname{Var}(v^T Bv)$, given $\mu = 0$ and $v^T Bv$ is unbiased for $\operatorname{tr}(B)$ (i.e., $\sigma^2 = 1$), we see that the variance is minimized if $\mathbb{E}(V^4)$ minimized. However, $\mathbb{E}(V^4) \geq \mathbb{E}(V^2)^2 = \sigma^4 = 1$, with equality (i.e., $\operatorname{Var}(V^2) = 0$) if and only if $V^2 = \mathbb{E}(V^2) = 1$ almost surely. This implies that $\mathrm{P}(V = 1) = p$, $\mathrm{P}(V = -1) = 1 - p$ $\implies \mathbb{E}(V) = 2p - 1 = 0$, so $p = \frac{1}{2}$ and V indeed follows the Rademacher distribution. □

The Monte-Carlo approximation derived from $\operatorname{tr}(B) = \mathbb{E}(v^T Bv)$ for v as in Theorem 3 (Corollary 4) is known as Hutchinson's trace estimator. Back to the calculation of the trace Jacobian in (7): to keep the dynamics deterministic within each call to the ODE solver, we may use a fixed noise vector v for every individual solve without introducing bias [1]:

$$\log p_\theta\big(T, y_i\big) = \log p_\theta\big(0, y(0, y_i)\big) - \int_0^T \operatorname{tr}\left(\frac{\partial f_\theta}{\partial y}\big(t, y(t, y_i)\big)\right) \mathrm{d}t \tag{8}$$

$$= \log p_\theta\big(0, y(0, y_i)\big) - \int_0^T \mathbb{E}_{v \sim V}\left[v^T \frac{\partial f_\theta}{\partial y}\big(t, y(t, y_i)\big) v\right] \mathrm{d}t \tag{9}$$

$$= \log p_\theta\big(0, y(0, y_i)\big) - \mathbb{E}_{v \sim V}\left[\int_0^T v^T \frac{\partial f_\theta}{\partial y}\big(t, y(t, y_i)\big) v \, \mathrm{d}t\right] \tag{10}$$

where we invoked Fubini's theorem in the last equality, knowing that the integrand is a bounded quadratic form which is absolutely integrable (recall f_θ's continuity conditions and that v has finite variance).

Comment. Using a single noise vector for the Monte-Carlo approximation of trace Jacobian suffices because training the CNF already averages the loss over a minibatch of data samples, which limits the variance in addition to having minimal estimator variance if v is Rademacher. Practically, exact trace calculation via autodifferentiation is typically tolerable in 2D, although it scales poorly in high dimension where Hutchinson's estimator actually becomes more advantageous.

Now for the punchline: the integrand of (10) may be computed in only $O(d)$ work. Indeed, denoting $J \coloneqq \frac{\partial f_\theta}{\partial y}\big(t, y(t, y_i)\big)$, we see that computing $v^T Jv$ involves two steps: computing the vector-Jacobian product $J^T v$ and taking the dot product $(J^T v)^T v$. The former can be done in reverse-mode autodiff in about the same order of work as one backpropagation through f_θ (namely $O(d)$), while the latter is simply a length-d dot product, so it likewise takes $O(d)$ work [3].

Why this question

This topic was chosen in the consideration that it merges several strands of the course and connects them to a modern machine learning paradigm: continuous normalizing flows bring together concepts from calculus, linear algebra, and statistical learning theory. At the same time, neural differential equation models are an active research area, and I am interested in exploring them further. In fact, this coursework felt like a useful stepping stone towards undertaking research opportunity such as a summer UROP in this field. Working through the derivations here also helped solidify core ideas from the module, especially the role of maximum likelihood estimation and statistical properties of estimation techniques.

Critical use of AI tools

AI tools were used in a limited and responsible way: they were used to help convert a hand sketch into a TikZ diagram (Figure 1) and to assist with locating and cross-checking relevant references in the flow-based modeling

literature. They were also used for light proofreading (spelling, grammar, and consistency of notation), with all mathematical derivations and final wording checked and edited manually.

Outlook

A natural next step is to complete the practical training loop of CNFs by incorporating the adjoint method for backpropagation through ODE solvers, and to study its theory such as memory-compute trade-offs in detail. Another direction is to examine CNFs in a concrete application setting (e.g. simulation modeling using real world data such as MNIST or CIFAR-10), including benchmarking against alternative generative models and evaluating their performances with appropriate statistical metrics (e.g. Wasserstein distances).

References

[1] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. *FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models*. arXiv:1810.01367 (2018).

[2] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. *Neural Ordinary Differential Equations*. arXiv:1806.07366 (2018).

[3] Patrick Kidger. *On Neural Differential Equations*. arXiv:2202.02435 (2022).

[4] M. F. Hutchinson. *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. Communications in Statistics—Simulation and Computation*, 19(2):433–450, 1990.

[5] Lecture 3: *Properties of MLE: consistency, asymptotic normality. Fisher information*. MIT OpenCourseWare 18.443 (Statistics for Applications), Fall 2006.

[6] Siva Balakrishnan. Lecture 12: *Properties of the MLE (Consistency)*. 36-700 (Probability and Mathematical Statistics I), Fall 2016, Carnegie Mellon University.